

EKF-SLAM with Unknown Correspondences

Georgios Soukaras

June 23, 2025

1 Introduction

Simultaneous Localization and Mapping (SLAM) is a core challenge in mobile robotics, requiring an agent to build a map of an unknown environment while concurrently estimating its own pose; the Extended Kalman Filter (EKF) is a popular solution thanks to its ability to handle non-linear motion and sensor models efficiently. In this project, we address the more difficult variant of EKF-SLAM with unknown correspondences, where the robot must resolve data associations on the fly—i.e., determine which landmark each noisy observation refers to—without prior labeling. To this end, we implement a Python-based simulator that models robot kinematics, sensor noise, and uncertainty propagation, and we employ the Mahalanobis distance as our statistical matching criterion, leveraging each landmark’s covariance to robustly decide whether an observation corresponds to an existing landmark or indicates a new one. This exercise not only demonstrates the practical integration of EKF and probabilistic data association but also highlights the importance of uncertainty-aware matching in achieving reliable mapping and localization in dynamic, real-world-like scenarios.

2 Motion Model

We employ the unicycle kinematic model to describe the motion of the robot. At time step t , the robot’s configuration is given by the state vector (x_t, y_t, θ_t) , where (x_t, y_t) denotes the position in the plane, and θ_t represents the orientation (heading angle). The robot moves with a linear velocity v_t and an angular velocity ω_t over a time interval of duration Δt .

Under these conditions, the robot’s state at the next time step, $(x_{t+1}, y_{t+1}, \theta_{t+1})$, is computed using the following kinematic update equations[2]:

$$\begin{pmatrix} x' \\ y' \\ \theta' \end{pmatrix} = \begin{pmatrix} x \\ y \\ \theta \end{pmatrix} + \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix}.$$

3 Data Association

As the robot navigates within a two-dimensional environment and observes various landmarks, it must be capable of distinguishing between distinct landmarks in order to maintain accurate data association.

To address this challenge, we utilize the *Mahalanobis distance* as a statistical association metric. This metric quantifies the dissimilarity between two measurement vectors, z and $\hat{z}$, by accounting for the underlying uncertainty in a non-Euclidean (covariance-weighted) space. The Mahalanobis distance provides a more robust measure than the standard Euclidean distance, particularly in the presence of noise and correlated measurement errors. We do this by taking into consideration the measurement covariance matrix Q of the measured sample vectors z and $\hat{z}$.

$$d_M(z, \hat{z}, Q) = \sqrt{(z - \hat{z})^T \cdot Q \cdot (z - \hat{z})} \quad (1)$$

By calculating the current observation's position and comparing it with the previously recorded positions of known landmarks, the system can determine whether the observed landmark corresponds to an existing entry in the map or represents a newly detected feature. Using the χ^2 distribution, we can set a probability threshold (e.g., 95% confidence) to accept or reject the hypothesis that the observation matches a known landmark. Also this χ^2 distribution has k degrees of freedom, where k is the dimensionality of the measurement (here is 2).

A hypothesis test is then performed:

- If $D^2 < \chi_{\alpha,k}^2$, the observation is considered a likely match to a known landmark (within confidence level α).
- If $D^2 \geq \chi_{\alpha,k}^2$, the observation is unlikely to match any known landmark and is treated as a new landmark.

4 EKF - SLAM with Unknown Correspondences

The Extended Kalman Filter - Simulation Localization and Mapping with unknown correspondences algorithm consists of 4 parts. The *prediction* of the robot's next state based on its motion model (lines 3 - 4). The *data association* of the landmarks that it's seeing at time t with the N_t landmarks that it has already noticed (lines 5 - 13). The *decision process* to either match the observed landmarks with existing ones (lines 14 - 16) or to create new ones and finally the *correction* or the update phase where the standard Kalman filtering is applied (lines 17 - 20).

Algorithm 1 SLAM Algorithm with Landmark Detection [1]

```
1: Data:  $(\mu_{t-1}, \Sigma_{t-1}), u_t, z_t, N_{t-1}$ 
2:  $N_t = N_{t-1}$ 
3:  $\bar{\mu}_t = g(u_t, \mu_{t-1})$ 
4:  $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$ 
5: for each  $z_t^i = (r_t^i, \phi_t^i)^T$  do
6:    $\begin{pmatrix} \bar{\mu}_{N_t+1,x} \\ \bar{\mu}_{N_t+1,y} \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{t,x} \\ \bar{\mu}_{t,y} \end{pmatrix} + \begin{pmatrix} r_t^i \cos(\phi_t^i + \bar{\mu}_{t,\theta}) \\ r_t^i \sin(\phi_t^i + \bar{\mu}_{t,\theta}) \end{pmatrix}$ 
7:   for  $k = 1$  to  $N_t + 1$  do
8:      $\hat{z}_t^k = \begin{pmatrix} \sqrt{(\bar{\mu}_{j,x} - \bar{\mu}_{t,x})^2 + (\bar{\mu}_{j,y} - \bar{\mu}_{t,y})^2} \\ \text{atan2}(\bar{\mu}_{j,y} - \bar{\mu}_{t,y}, \bar{\mu}_{j,x} - \bar{\mu}_{t,x}) - \bar{\mu}_{t,\theta} \end{pmatrix}$ 
9:      $H_t^k = h_t^k F_{x,k}$ 
10:     $S_t^k = H_t^k \bar{\Sigma}_t [H_t^k]^T + Q_t$ 
11:     $\pi_k = (z_t^i - \hat{z}_t^k)^T [S_t^k]^{-1} (z_t^i - \hat{z}_t^k)$ 
12:   end for
13: end for
14:  $\pi_{N_t+1} = \alpha$ 
15:  $j(i) = \arg \min_k \pi_k$ 
16:  $N_t = \max\{N_t, j(i)\}$ 
17:  $K_t^i = \bar{\Sigma}_t [H_t^{j(i)}]^T [S_t^{j(i)}]^{-1}$ 
18:  $\bar{\mu}_t = \bar{\mu}_t + K_t^i (z_t^i - \hat{z}_t^{j(i)})$ 
19:  $\bar{\Sigma}_t = (I - K_t^i H_t^{j(i)}) \bar{\Sigma}_t$ 
20:  $\mu_t = \bar{\mu}_t$  and  $\Sigma_t = \bar{\Sigma}_t$ 
21: return  $(\mu_t, \Sigma_t)$ 
    =0
```

4.1 Prediction Phase

The first phase consists of updating how many distinct landmarks the robot has identified and added to its map by time t . Then using the motion model with the current inputs and the last position $g(v_t, \mu_{t-1})$ we create a new *estimate* of the robots position. Additionally it updates the covariance matrix Σ_t using Jacobian of the motion model

$$G_t = \left. \frac{\partial f}{\partial x} \right|_{\substack{x=\mu_{t-1} \\ u=u_t}}.$$

4.2 Data association

Here is where the algorithm gets sophisticated. For each observed landmark z_t^i e robot calculates its *Mahalanobis distance* to each of the N_t landmarks already on the map.

- Step 6: the robot computes where z_t^i would lie in world-frame coordinates if it were treated as a new landmark. This hypothetical position is denoted $\bar{\mu}_{N_t+1}$.
- Step 8: we perform the inverse of step 6. Here, $\hat{z}_k$ represents what the sensor *should* read if it is looking at landmark k .

- Step 9: the Jacobian of the observation model h_i^k is calculated. It captures how small changes in the robot's state and landmark k 's position would affect the expected observation. The F_i^k is the selection matrix in order to pick only the robot autocovariance matrix. In our case we only care about the r, ϕ of our measurements. So H_I^K will be:

$$H = \frac{1}{q} \begin{bmatrix} -\sqrt{q} \Delta x & -\sqrt{q} \Delta y & 0 & \sqrt{q} \Delta x & \sqrt{q} \Delta y \\ \Delta y & -\Delta x & -q & -\Delta y & \Delta x \end{bmatrix}$$

- Step 10: the *Innovation Covariance Matrix* is calculated as the sum of the uncertainty due to linearization and the sensor noise Q_t .
- Step 11: the robot computes the actual Mahalanobis distance π_k . This measures how different the actual observation is from what we'd expect if looking at landmark k . A small π_k means the observation matches well with landmark k ; a large π_k means it's an unlikely match.

4.3 Decision Process

- Step 14: we set

$$\pi_{N_t+1} = \alpha,$$

which is the probability threshold for creating a new landmark. The parameter α encodes how conservative the algorithm is about adding new landmarks versus matching existing ones.

- Step 15: we select the most likely association by finding the index k that minimizes π_k . This k may correspond to an existing landmark ($k \leq N_t$) or the “new landmark” option ($k = N_t + 1$).
- Step 16: we update the landmark count. If the best association is the new-landmark option ($j(i) = N_t + 1$), we increment N_t to add the new landmark to the map.

4.4 Correction Step

Once a decision has been made—either to associate the observation with an existing landmark or to introduce a new landmark—the Kalman Filter correction is applied to update the robot's belief.

For each hypothesis, and for each valid observation z_t^i associated with landmark $j(i)$, the following update is performed:

- Step 17, compute the Kalman gain:

$$K_t^i = \bar{\Sigma}_t [H_t^{j(i)}]^T [S_t^{j(i)}]^{-1}$$

- Step 18, update the state estimate:

$$\bar{\mu}_t = \bar{\mu}_t + K_t^i (z_t^i - \hat{z}_t^{j(i)})$$

- (Normalize the heading angle):

$$\bar{\mu}_{t,\theta} = \text{wrap_angle}(\bar{\mu}_{t,\theta})$$

- Step 19, update the covariance estimate:

$$\bar{\Sigma}_t = (I - K_t^i H_t^{j(i)}) \bar{\Sigma}_t$$

These steps reduce the uncertainty in the robot's state and the positions of the associated landmarks. The angle normalization ensures that the heading remains within the range $[-\pi, \pi]$.

After all observations have been processed, the corrected state and covariance become the new belief:

$$\mu_t = \bar{\mu}_t, \quad \Sigma_t = \bar{\Sigma}_t$$

This completes one iteration of the EKF-SLAM update loop.

4.5 Motion with Repulsion-Based Navigation

When the robot is inside the defined box(meaning box in which lay all landmarks), it follows a sweeping path with repulsion from nearby landmarks.

Let (x, y, θ) be the current robot pose, and (l_x, l_y) be the position of a landmark. The repulsion vector is calculated as:

$$\mathbf{r} = \sum_{\|\mathbf{l}-\mathbf{p}\| < d_{\text{rep}}} -\frac{\mathbf{l}-\mathbf{p}}{\|\mathbf{l}-\mathbf{p}\| + \epsilon}$$

where: - $\mathbf{p} = [x, y]^T$ is the robot position, - $\mathbf{l} = [l_x, l_y]^T$ is the landmark position, - $d_{\text{rep}} = 2.0$ is the repulsion threshold, - $\epsilon = 10^{-3}$ avoids division by zero.

The desired movement vector is a combination of goal direction $\mathbf{g}$ and repulsion:

$$\mathbf{h} = \mathbf{g} + \alpha \cdot \mathbf{r}$$

where $\alpha = 1.5$ is a scaling factor for repulsion.

The desired heading is:

$$\theta_d = \text{atan2}(h_y, h_x)$$

Then, the angular control is:

$$\Delta\theta = \text{wrap}(\theta_d - \theta)$$

$$w = \text{clip}(\Delta\theta, -\theta_{\max}, \theta_{\max})$$

$$v = 1.0$$

with $\theta_{\max} = 30^\circ$.

The control input is finally(which is parsed as input-control in ekf-slam algorithm):

$$\mathbf{u} = \begin{bmatrix} v \\ w \end{bmatrix}$$

5 Multi-Hypothesis Tracking (MHT)

To handle ambiguity in landmark association, we implement a Multi-Hypothesis Tracking (MHT) strategy within the EKF-SLAM framework. Instead of committing to a single data association decision at each time step, as described in the algorithm before, the refined algorithm maintains multiple concurrent hypotheses, each representing a different possible interpretation of the observations.

Each hypothesis contains:

- The robot and landmark state vector μ
- The corresponding covariance matrix Σ
- A cumulative score, reflecting the hypothesis' overall likelihood based on Mahalanobis distance penalties

During the update step, for each measurement:

1. The algorithm computes Mahalanobis distances to all existing landmarks in a given hypothesis.
2. For each valid association (distance below a chosen threshold), a new hypothesis is created by cloning the parent and applying the Kalman correction for that association.
3. If no valid associations exist, a new hypothesis is created with a new landmark appended to the state.

This branching mechanism creates a hypothesis tree over time, where each leaf node represents a distinct interpretation of the robot's environment and measurements.

Hypothesis Scoring and Pruning

Since the number of hypotheses can grow rapidly, we apply a pruning mechanism:

- After processing all observations at a given timestep, hypotheses are sorted by their score.
- The score for each hypothesis is decremented by the Mahalanobis distance of its matched observations (lower is better).
- Only the top K hypotheses (e.g., $K = 5$) are retained for future processing; the rest are discarded.

This pruning ensures computational efficiency while preserving the most promising interpretations. The hypothesis with the highest score is used for state output and visualization.

6 Landmark Merging via Mahalanobis Distance

Over time, especially in environments with repeated structures or noisy observations, the system may create multiple landmarks representing the same physical location. To mitigate this and keep the map compact and semantically correct, we apply a post-correction merging process based on Mahalanobis distance.

When Merging Occurs

After the correction step and hypothesis pruning, we apply merging only to the best hypothesis (i.e., the one with the highest score). This ensures that merging decisions are made based on the most reliable interpretation of the environment.

Merging Criteria and Method

For every pair of landmarks i and j in the best hypothesis, we compute the squared Mahalanobis distance between their positions:

$$D_{ij}^2 = (\mu_i - \mu_j)^T (S_i + S_j)^{-1} (\mu_i - \mu_j)$$

Where:

- μ_i, μ_j are the 2D means (positions) of the landmarks,
- S_i, S_j are their respective 2×2 covariance matrices,
- $(S_i + S_j)$ captures total uncertainty between the pair.

If $D_{ij}^2 < \delta_{\text{merge}}$, we consider the landmarks to be statistically indistinguishable and merge them. In our implementation, we choose:

$$\delta_{\text{merge}} = 4$$

This threshold is more conservative than the association threshold (9.21), ensuring only clearly redundant landmarks are combined.

Merged Landmark Update

The merged landmark replaces the original pair with:

- Mean: the average of the two positions, $\mu_{\text{merged}} = \frac{1}{2}(\mu_i + \mu_j)$
- Covariance: the average of the two covariances, $S_{\text{merged}} = \frac{1}{2}(S_i + S_j)$

The state vector and covariance matrix are accordingly adjusted to reflect the new landmark configuration.

Purpose and Impact

Merging improves map compactness, prevents over-segmentation, and enhances the quality of long-term mapping. Since merging is based on Mahalanobis distance, it naturally incorporates uncertainty, making the decision robust even in noisy scenarios.

References

- [1] Stanford lectures, 2022.
- [2] Wolfram Burgard Sebastian Thrun, Dieter Fox. *PROBABILISTIC ROBOTICS*.
Massachusetts Institute of Technology, 2006.